Telematics Platform User Guide

Application Permissions Manager

Document Information	
Filename	TP_F101U-00025_User_Guide_Application_Permissions_Manager
Status	Release
Revision Date	Mar 16, 2021
Version	03.00
Document ID	TP_F101U-00025

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Table of contents

1 Introduction 4
1.1 Notational Conventions 5
2 Setup 5
2.1 Tools/Equipment Needed 5
2.2 Setup Instructions 5
2.3 Setup Troubleshooting 8
3 Features/Components 8
3.1 Permission policy 8
3.1.1 Overview 8
3.1.1.1 Platform Content Overview 8
3.1.1.2 Product Expectations 8
3.1.1.3 Extending Platform Functionality 9
3.1.2 Setup 9
3.1.2.1 Tools/Equipment Needed 9
3.1.2.2 Setup Instructions 9
3.1.2.3 Setup Troubleshooting 9
3.1.3 Usage 9
3.1.3.1 Basic Instructions 9
3.1.3.2 Detailed Instructions 9
3.1.3.3 Testing/Error Reporting 16
3.2 Authorization Policy 17
3.2.1 Overview 17
3.2.1.1 Platform Content Overview 17
3.2.1.2 Product Expectations 17
3.2.1.3 Extending Platform Functionality 17
3.2.2 Setup 17
3.2.2.1 Tools/Equipment Needed 17
3.2.2.2 Setup Instructions 17
3.2.2.3 Setup Troubleshooting 18
3.2.3 Usage 18
3.2.3.1 Basic Instructions 18
3.2.3.2 Detailed Instructions 18
3.2.3.3 Testing/Error Reporting 19
3.3 Updating Authorization status 20
3.3.1 Overview 20
3.3.1.1 Platform Content Overview 20
3.3.1.2 Product Expectations 20
3.3.1.3 Extending Platform Functionality 20
3.3.2 Usage 20

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.3.2.1 Basic Instructions 20
3.3.2.2 Detailed Instructions 20
3.3.2.3 Testing/Error Reporting 21
4 Known Issues 22
5 FAQs 22
6 Troubleshooting/Tips and Tricks 22
7 References 22
7.1 Abbreviations/Acronyms/Definitions 22
7.2 Platform Documentation 23
7.3 Website links 23
8 Revision History 24

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides usage information for Application Permissions Manager.

The Application Permissions Manager is an engine that provides interfaces to manage **authorization and permissions** to access arbitrary resources (like voice, packet data etc.) by different services. **The Application Permissions Manager does not grant access to the physical resources.**

It also provides the flexibility to allow the underlying policy (authorization and permission) logic to be swapped out completely if a project team wants to include a proprietary policy algorithm.

For detailed understanding on the commonly used terms like Applications, Resource, etc., please refer to the Section 7.1.

A client of APM can request for a resource and depending on its authorization and permission priority it will gain the requested resource and also may be denied of it, likewise if needed, the service already holding the requested resource will have its permission revoked.

This document will cover details about features provided by this service, configuration usage, and dynamic behavior. The target audience for this document are developers, testers and architects who will use the Telematics Platform for Application Permissions Manager service.

Below diagram gives high level understanding of Application Permissions Manager (APM). Here, Application requests for permission through APM. APM in turn checks whether application is authorized to get access and then returns permission back to application. Now, application may decide to delegate this guaranteed policy token to service, and there on the service can access the underlying resource based on received token.

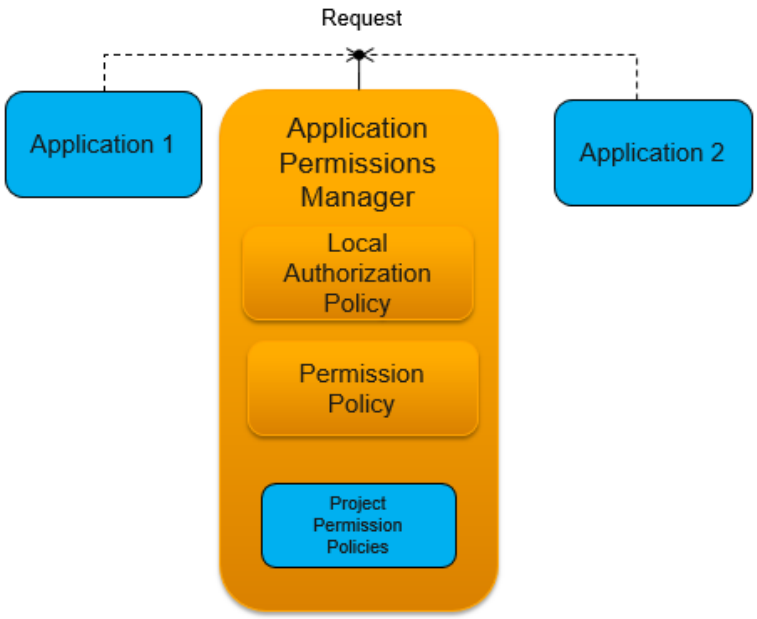


Figure 1: System Context Diagram

APM supports **two configuration modes** (“Exclusive” or “Shared" resources between application processors). For detailed information and setup steps, please refer to Section 2.2.

NOTE: The Application Permissions Manager does not grant access to physical resources.

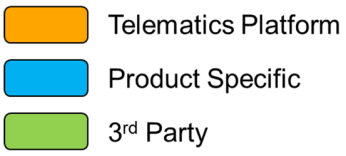
© Continental Automotive GmbH 2019
Confidential and Proprietary

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

2.1 Tools/Equipment Needed

Not applicable.

2.2 Setup Instructions

The Configuration file – cfg.json

1) **Overview:** The platform maintains a file that is a record of the resource priority configurations set by product development team. The product development team can add or modify the configurations stored in this file. The configuration thus is kept as a json file called '**cfg.json'** and is available at build time.

   Points to be noted:
   a. Contents of cfg.json has to be defined by product team.
   b. Only a single configuration file is supported currently.
   c. Clients are trusted and will obey and not modify the returned permissions.
   d. The file cannot be bigger than 100KB.

2) **Contents:** The resource – priority configuration file contains the following:

   application name (string)
   permission name (string, zero or more permissions)
   resource name (string, one or more resources associated to the permission)
   resource priority (integer, an application priority for the resource)
   authorization status (string)

© Continental Automotive GmbH 2019
Confidential and Proprietary

The cfg.json will typically have the following notation:

```
{
    "name": "DIAG",
    "permission": [
        {
            "name" : "Allowed to Communicate",
            "resource": {
                "name": "packetdata",
                "priority" : 2
            }
        },
        {
            "name": "perm2",
            "resource": {
                "name": "res1",
                "priority": 3
            }
         }
    ],
    "auth": true
},
```

Application/Client name

Permission name

Resource name

Resource priority value

Authorization status

Here is a tabular form of the above json with a common list of clients of APM and corresponding permissions for resources along with their priorities.

Clients	Allowed To Communicate (Permission)		
	'Voice' resource	'Packet Data' resource	'Over Temp' Resource
ACN	2		
SOS	4		
ICALL	5		
Stolen Vehicle	2		1
Locator		2	
Alarm		2	
Remote Control		2	
Provisioning		2	
Over Temp	3	1	1
Diagnostics	1	1	2

Table 1: Clients, corresponding permission, resources and priorities

© Continental Automotive GmbH 2019
Confidential and Proprietary

How to be a Client of Application Permissions Manager

1) Use pkg-config in the client application Makefile to link to the APM client library

```
LDFLAGS += `pkg-config --libs libcas-apm`
CFLAGS += `pkg-config --cflags libcas-apm`
```

2) Include Application Permissions Manager header file in client application source code

```
#include "application_permissions_mgr.h"
```

3) Wait for Application Permissions Manager to be ready at startup.

a. If client application has a "hard" dependency on Application Permissions Manager to be running use systemd .service "After" field

```
[Unit]
After=apm.service
```

b. If client application has only a single thread (or portion of process) that is dependent on Application Permissions Manager non-blocking TP service ready API:

```
tpSVCMON_registerServiceStateChange(APM_SERVICE_NAME, FALSE,
callback, &monitor_id);
```

4) **Security:** Usage of Application Permissions Manager APIs require clients (user) to be a member of **"apm"** group.

5) Once Application Permissions Manager service is ready, initialize communication with the Application Permissions Manager in client application source code (**note:** pre-requisite is that the client must first call `tpSYS_initIPC()`)

```
tpAPM_Error_t apm_ret = E_APM_ERROR_NONE;
apm_ret = tpAPM_init ();
/* check return code */
```

Choose the right APM mode for your project – "Exclusive" or "Shared" Configuration

1) **Context:** Initial implementation of APM in a dual core architecture utilized independent APM services in each core managing exclusive resources. Each service has a unique configuration file: one *cfg.json* file with apps and permissions for one core, and one *cfg.json* file for the other core. This APM use case does not allow the notion of "shared" resources between cores.

2) **Compile flag:** A new compile flag enables this shared configuration mode (available in *<$project>/config/board* file for Primary AP configurations):

```
CAS_APM_REMOTE_SHARED_CONFIGURATION
= 0 (disabled) | = 1 (enabled)
```

With this variable set to 1 before build steps, APM service will be used remotely by clients running on Primary AP, meaning that **only one APM daemon will run on Secondary AP** side and **manage permissions for both processors**.

© Continental Automotive GmbH 2019
Confidential and Proprietary

To illustrate a basic scenario using this remote shared configuration, a sequence chart is available in Section 3.2.3.2.1.1, Figure 5.

On the contrary, if this variable is set to 0 (default behavior), an APM daemon will run on both processors, and each instance will deal with its own resources and permissions independently.

3) **Service Monitor:** To manage APM service states, the Service Monitor component is necessary - please refer to TP_F101U-00023 for implementation guidelines. Clients on the Primary AP need to wait for APM service to be marked as "*READY*" by Service Monitor before accessing the remote APM service. Additionally, link failures can occur between processors, as detailed in Section 6.

2.3 Setup Troubleshooting

Not applicable

3 Features/Components

3.1 Permission policy

3.1.1 Overview

3.1.1.1 Platform Content Overview

Permission definition: A permission is used to manage priorities of applications that compete for limited resources or meet OEM-specific priority requirements.

A permission is returned in a token to the requesting application.

The Application Permissions Manager uses application priorities and the current state of all application permissions granted to determine the requesting application permissions. The Application Permissions Manager uses resources that can be configured and an application's priority on these resources to determine application permission. A permission as a token is passed back to the application indicating if the permissions are granted or not granted so the application can determine if it is ok to proceed with further necessary actions.

Example use cases are in mentioned in Section 3.1.3.2.2.1 and Section 3.1.3.2.2.2.

3.1.1.2 Product Expectations

Product is expected to provide the policies that govern the management of the permissions of different application. These policies are realized by APM from the configuration file (cfg.json). For a detailed understanding on the configuration file refer to "The Configuration file – cfg.json" of Section 2.

- Product needs to register to receive the status change notifications. This in turn helps the product to remain updated with the status of the permission of the resources.
- The Application Permissions Manager assumes that the applications requesting permissions are trusted applications and will obey and not modify the returned permissions. Product needs to act according to the permissions returned (for example to notify release if permissions are revoked). For more details refer to the use case diagrams in Section 3.1.3.2.1.1.
- APM does not enforce that the client actually releases permission (in case permission is revoked). A delegated service can perform this function. Nevertheless, when APM revokes permission to access a certain resource, it is expected that the product releases permissions for the same (for details refer Figure 2).
- When any application is not granted permission to access a certain resource by APM (may be due to it being of lower priority), product must wait for the APM to notify that the resource is available to request for it again (for details refer Figure 3). The product is not allowed to use the resource until APM notifies about the permission being released.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.1.3 Extending Platform Functionality

› The priorities of the application are usually positive integers starting from 1. The higher the priority the lower the integer. While there is only one exception of '-1' priority, indicating the client having such priority can always apply the resource successfully to permission determination, without affecting the system's current status.

3.1.2 Setup

Not applicable.

3.1.2.1 Tools/Equipment Needed

Not applicable.

3.1.2.2 Setup Instructions

Not applicable.

3.1.2.3 Setup Troubleshooting

Not applicable.

3.1.3 Usage

3.1.3.1 Basic Instructions

One or more **resource(s)** are used to determine whether a permission is granted (or revoked). An application may have a priority defined for each resource associated to a permission. The resources can be defined by each application team per the application needs. An application priority is compared to other priorities of applications currently granted the respective permission to determine the applications permission status. If any of the checks on priority of the resource fail, then the application is not granted the permission. The application receives from the Application Permissions Manager a granted status of all permissions required. An application that has been granted permissions previously may have the permissions revoked at a later time if a higher priority application has requested the same permission. The application is asked to relinquish the permissions, "a trusted application". The application stops its use of granted permissions. A priority defined equal to -1 means the requesting application always receives the resource but does not interrupt other applications.

3.1.3.2 Detailed Instructions

References to Application Permissions Manager APIs and types that can be found in this section, for more details on these APIs and types see the Doxygen links in the following table:

API/Type Description	Doxygen Path

Determines if application configured permissions can be granted.	`Modules`
	`|-> Security`
	`|-> Application Permissions Manager (APM)`
	`|-> tpAPM_requestPermissions`

Releases the previously granted permissions.	`Modules`
	`|-> Security`
	`|-> Application Permissions Manager (APM)`
	`|-> tpAPM_releasePermissions`

Checks the currently granted permissions for this application.	`Modules`
	`|-> Security`
	`|-> Application Permissions Manager (APM)`
	`|-> tpAPM_checkPermissions`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Registers to be notified if there is a change in permissions.	Modules |-> Security |-> Application Permissions Manager (APM) |-> **tpAPM_registerPermissionsStatusChange**
Deregisters to not be notified anymore if there is a change in permissions.	Modules |-> Security |-> Application Permissions Manager (APM) |-> **tpAPM_deregisterPermissionStatusChange**

Table 2: Doxygen paths for permission related APIs

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.3.2.1 Permission Handling

The APM supports the following permission granted status, which are indications for the type of access that was returned by APM (for a request) and given up by the client (for a release).

```
E_APM_NOT_GRANTED
E_APM_GRANTED
E_APM_GRANTED_ON_RELEASE
```

3.1.3.2.1.1 Sequence Diagrams

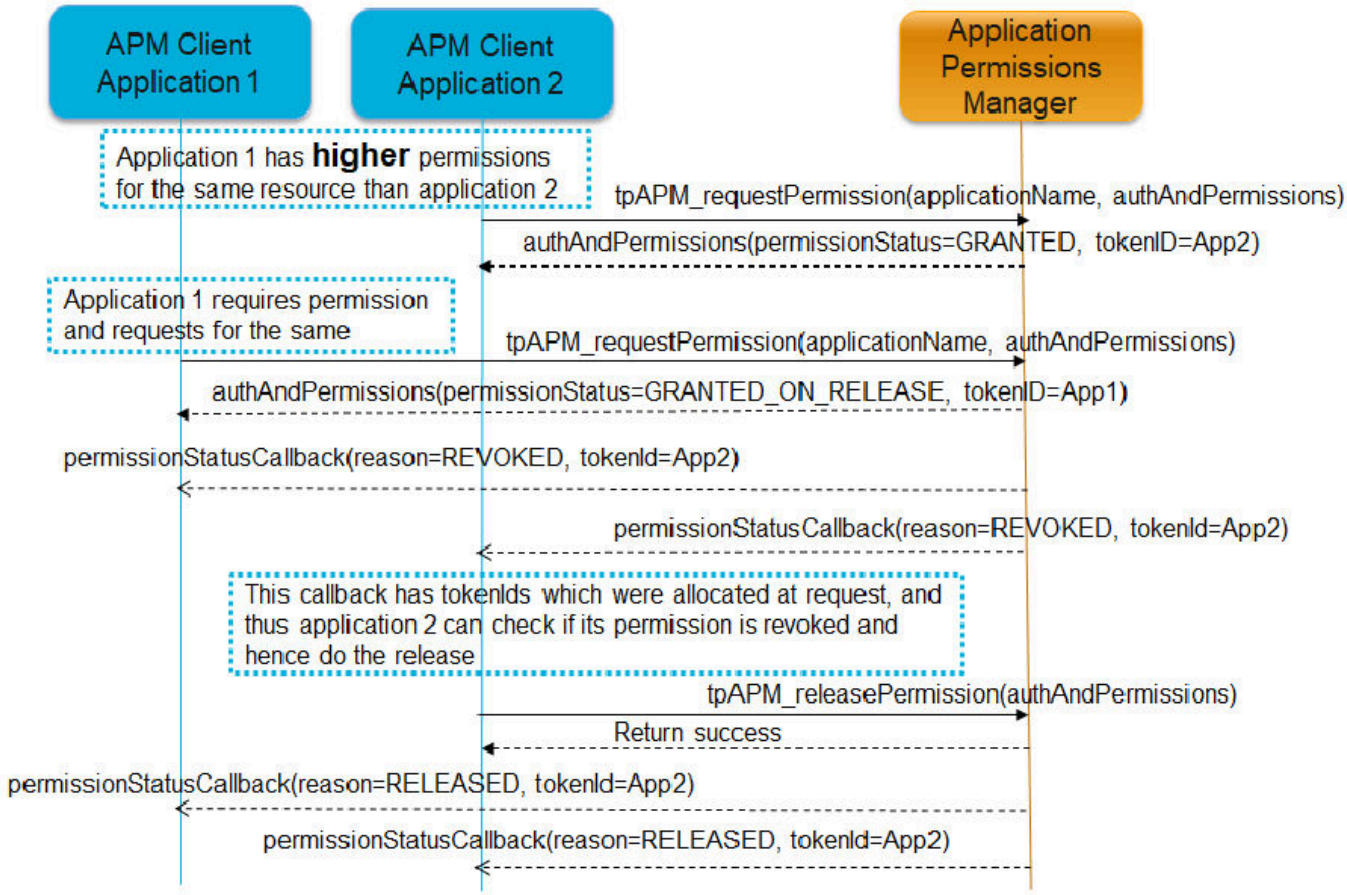


Figure 2: Handling authorization permissions request from higher priority application

© Continental Automotive GmbH 2019
Confidential and Proprietary

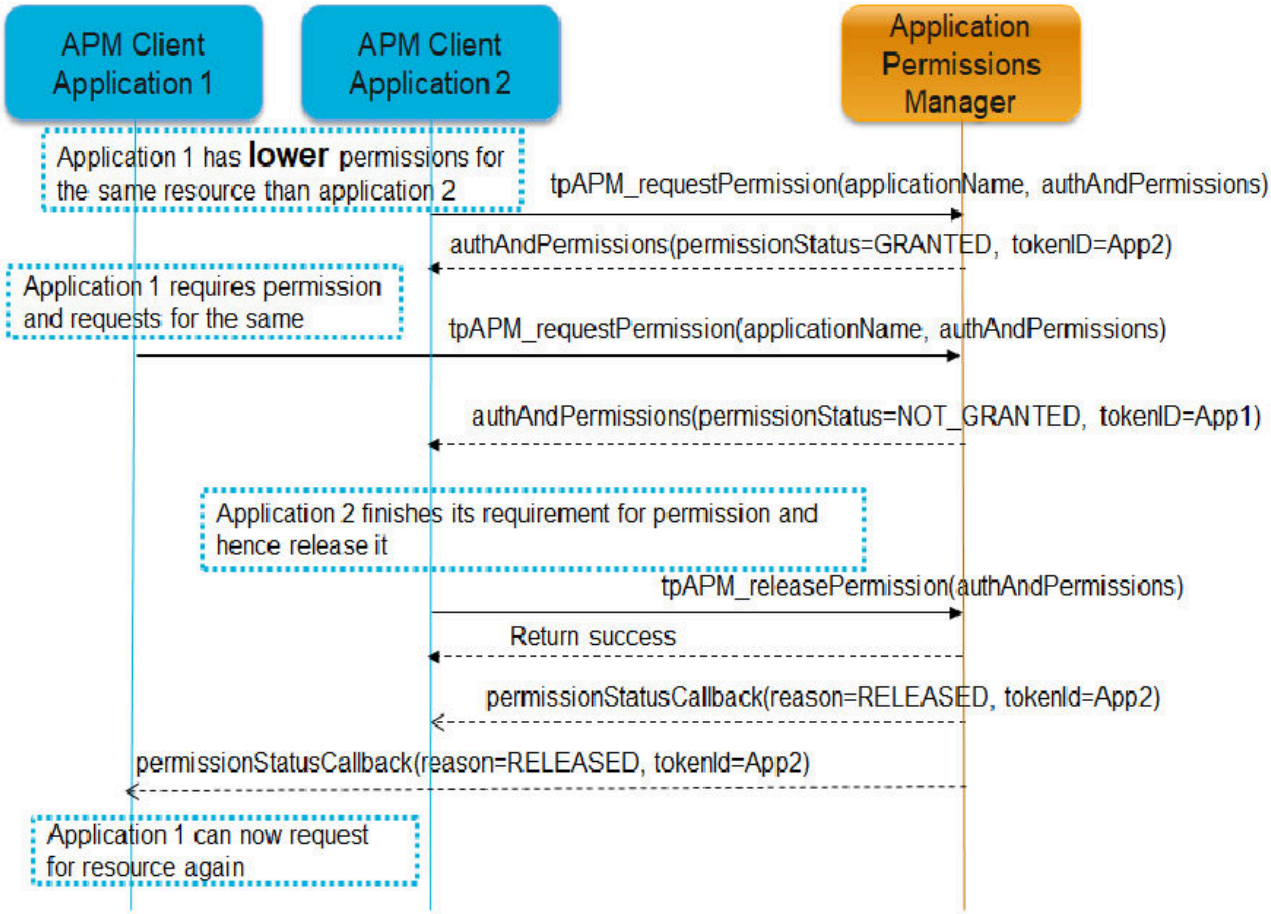


Figure 3: Handling authorization permissions request from lower priority application

© Continental Automotive GmbH 2019
Confidential and Proprietary

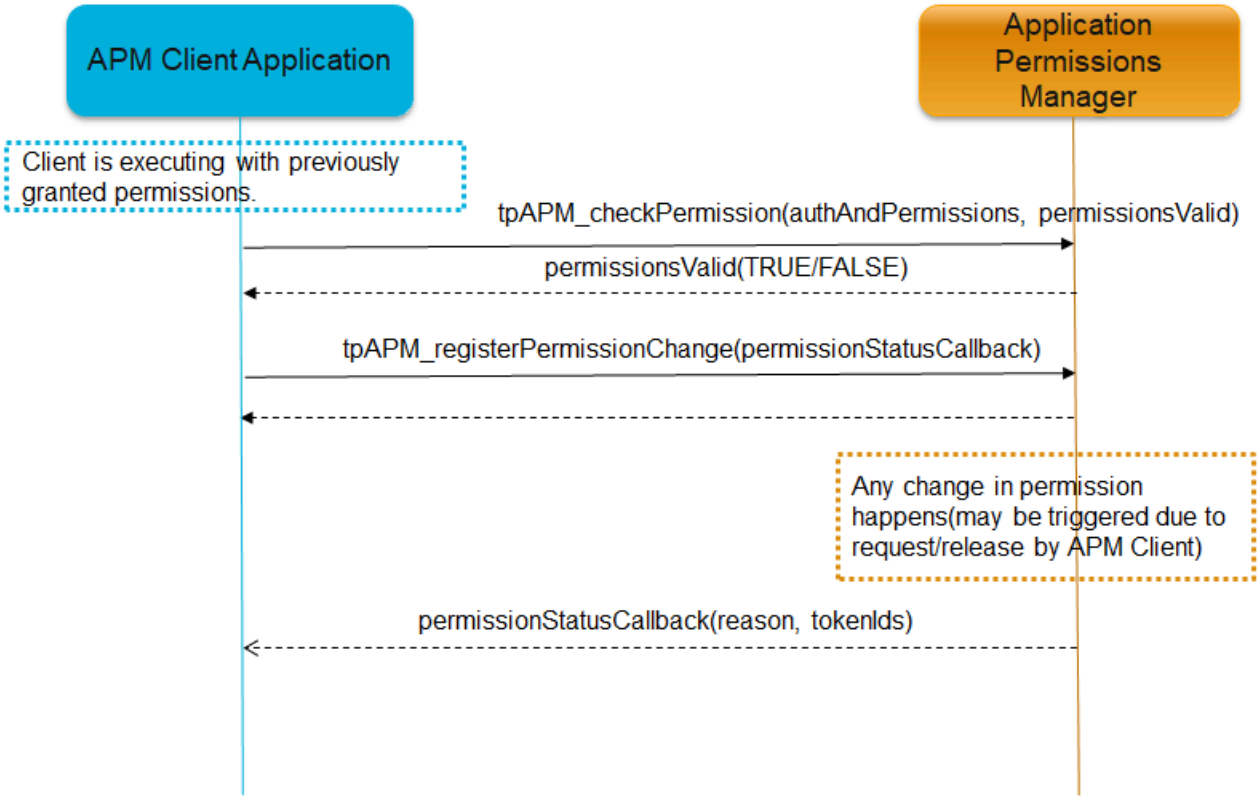


Figure 4: Check and notify change in permissions

© Continental Automotive GmbH 2019
Confidential and Proprietary

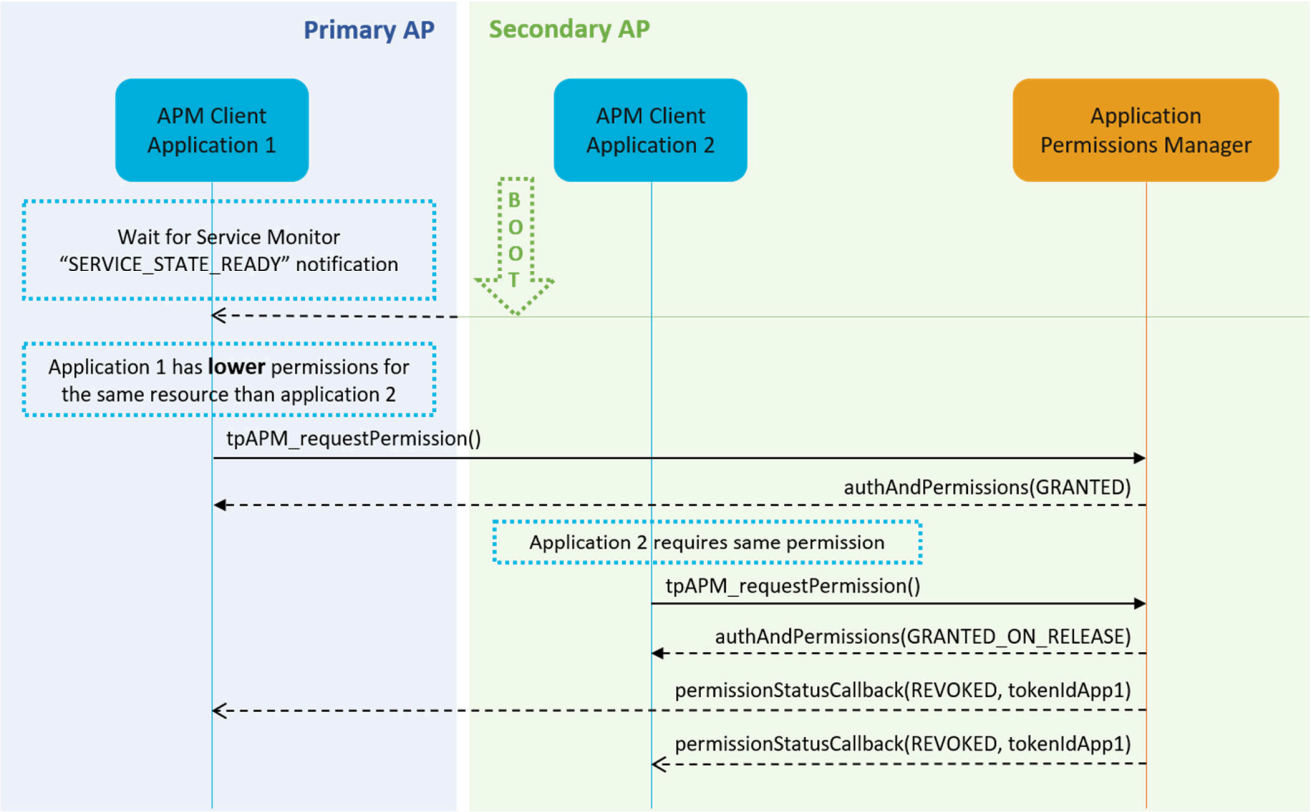


Figure 5: A basic use case scenario in remote shared configuration

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.3.2.1.2 Usecase 1: Generic permission request handling between

› The project team can specify rules for permissions that the project requires (details about configurations can be seen in "The Configuration file – cfg.json" of Section 2). Here there are two types of resources that are defined to control whether the "allowed to communicate" permission is granted.

Application	Allowed To Communicate (permission)	
	'Voice' Resource	'Packet Data' Resource
Voice 1	2	
Voice 2	3	
Voice 3	4	
Packet Data 1		2
Packet Data 2		2
Diagnostics	1	1

› In this example,
  › The Diagnostics application request revokes all other applications that were previously granted permissions, i.e. revokes the "allowed to communicate" permission.
  › The package data applications can run in parallel with the voice applications.
  › The voice applications 1, 2, 3 have permissions that allows voice 1 application to be granted the "allowed to communicate" permission over voice application 2 and 3.

3.1.3.2.1.3 Usecase 2: Stolen Vehicle Tracking Request

Below is the configuration policy used by Application Permissions Manager.

Application	Allowed To Communicate (permission)	
	'Voice' Resource	'Packet Data' Resource
Locator		2
ICALL	5	
Stolen Vehicle Tracking	2	

› Preconditions(for details about applications and resources' priority refer Section 7.1 and for sequential understanding refer to Figure 2)
  › Locator is currently "allowed to communicate" over "Packet Data" resource with priority 2.
  › ICALL is currently "allowed to communicate" over "Voice" resource with priority 5.
› Use Case
  › Stolen Vehicle Tracking requests permissions from Application Permissions Manager using the `tpAPM_requestPermission` API.
  › ICALL has lower priority than stolen vehicle tracking.
  › Stolen Vehicle Tracking does not affect packet data applications that are running.

© Continental Automotive GmbH 2019
Confidential and Proprietary

- Conclusion: Stolen Vehicle should be “allowed to communicate”
  - Action:
    1. APM returns granted_on_release “allowed to communicate” to Stolen Vehicle Tracking application (and also other registered applications) as the return value in the `tpAPM_requestPermission` API call from Stolen Vehicle Tracking application.
    2. APM notifies permissions revoke “allowed to communicate” to ICALL application (and also other registered applications) by triggering the `permissionsStatusCallback` registered when the ICALL client had called the `tpAPM_registerPermissionsStatusChange` API.
  - Expectation: ICALL needs to call `tpAPM_releasePermissions` API on receiving the above permission change notification with permission as revoked and corresponding tokenIds.
  - Action: On receiving release of permissions from ICALL, APM notifies permissions released “allowed to communicate” to ICALL application (and also other registered applications) by triggering the `permissionsStatusCallback`.
  - Action: APM internally searches the applications whether any of them can be allocated the now available “Voice” resource by checking whether they are having permission as granted_on_release. Thus, Stolen Vehicle Tracking acquires the resource.

NOTE: These permissions, priority and 'allowed' are completely unrelated to features of the Operating System and the actual state of an Application's process or thread. Also, these permission using Application Permissions Manager APIs do not let the client access physical resources directly.

3.1.3.3 Testing/Error Reporting

The errors returned by the APM is as in the following Doxygen path.

Type Description	Doxygen Path

This enum defines the types of application permission manager library errors	`Modules`
	`|-> Security`
	`|-> Application Permissions Manager (APM)`
	`|-> Enumerations`
	`|-> tpAPM_Error_t`

Table 3: Doxygen paths for APM error enum

Point to be noted:

- If any of the checks on priority of the resource fail, then the application is not granted the permission.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2 Authorization Policy

3.2.1 Overview

3.2.1.1 Platform Content Overview

Authorization definition: The application must be allowed to activate. Authorization is always a prerequisite to permission, i.e., a client cannot be allowed to access a resource if it is not authorized to access it.

For example, has the vehicle owner paid for service? (Refer to column 1 of Table 4 in Section 3.3.2.2 for a list of such services)

3.2.1.2 Product Expectations

- Authorization is determined from the policies in the config file, hence the cfg.json file needs to be correctly set.
- The platform Application Permissions Manager listens to application authorization changes and will notify applications currently granted permissions if the authorization state changes. Product needs to register to receive the status change notifications. This in turn helps the product to remain updated.
- Authorization updates are handled by an entity independent of the platform Application Permissions Manager. These updates can be done using persistence APIs by clients who are in 'apm' group. Product needs to be aware that such modifications have to be handled by it by being in 'apm' group.
  For example: Individual authorization for applications to run is maintained locally, this is stored in persistency. In this case, the authorization information is stored in a json file called cfg.json. This file's (cfg.json) location is stored in persistency. And according to product teams' requirements this file can be extended/modified. For complete details about the configuration file, refer to "The Configuration file – cfg.json" of Section 2.

3.2.1.3 Extending Platform Functionality

- The authorization information is stored in a json file called cfg.json in the following format.

```
{
    "name": "DIAG",
    "permission": [{
                    "name" : "Allowed to Communicate",
                    "resource" : {
                                    "name": "packetdata",
                                    "priority" : 2
                    }
                }],
    "auth": true
},
```

As shown in the above configuration, the application "DIAG" has the authorization to have the permission "Allowed to Communicate" for "packetdata" resource at priority 2.

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not applicable.

3.2.2.2 Setup Instructions

Not applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.2.3 Setup Troubleshooting

Not applicable.

3.2.3 Usage

3.2.3.1 Basic Instructions

Most Telematics Systems include this authorization requirement for its Applications.

In the telematics system, there are few applications that are offered by the product team in the form of features. For a list of such features please refer to first column of Table 1. These features can be built-in by the product team by keeping the configurations available in the cfg.json file. But whether these applications are “activated” or not is decided by the product team based on conditions defined by the product team itself.

As for example: Whether an application is “allowed to activate” at all is decided if the vehicle owner has paid for this service or not? If the vehicle owner decides to pay for activating the service “SOS”, then the config file is modified and the auth_status for “SOS” application is set to “TRUE”.

Also, the authorization status can be checked by the clients at any point of time using APM library APIs.

3.2.3.2 Detailed Instructions

References to Application Permissions Manager APIs and types that can be found in this section, for more details on these APIs and types see the Doxygen links in the following table:

API/Type Description	Doxygen Path

Initialize communication with the Application Permissions Manager	`Modules`
	`|-> Security`
	`|-> Application Permissions Manager (APM)`
	`|-> tpAPM_init`

Checks the authorization status for the given application.	`Modules`
	`|-> Security`
	`|-> Application Permissions Manager (APM)`
	`|-> tpAPM_checkAuthStatus`

Table 4: Doxygen paths for authorization related APIs

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.3.2.1 Check authorization status

If the client application wants to know only the authorization status of an application, it should call the `tpAPM_checkAuthStatus(applicationName, authStatus)` API instead of requesting permissions. And based on the status (TRUE/FALSE) it can decide to request permission or not.

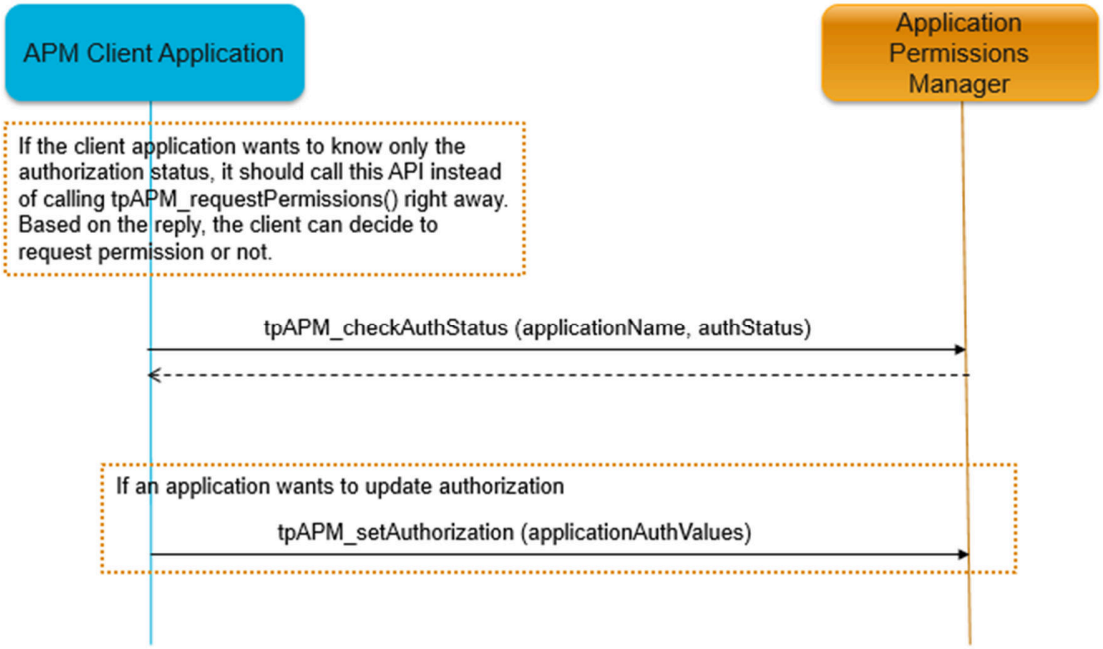


Figure 6: Check authorization status

3.2.3.3 Testing/Error Reporting

Same as mentioned in Section 3.1.3.3.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.3 Updating Authorization status

3.3.1 Overview

3.3.1.1 Platform Content Overview

It is required that the platform maintains a file that is a record of the resource priority configurations set by product development team. And that the record can be updated whenever the product development team deems necessary.

3.3.1.2 Product Expectations

Permission and authorization policies in the cfg.json can be updated using the Application Permissions Manager APIs directly by clients given that they are registered with 'apm' group.

3.3.1.3 Extending Platform Functionality

Not applicable.

3.3.2 Usage

3.3.2.1 Basic Instructions

This feature allows product development team to modify the authorization policies. For example: A user has recently paid to use the SOS service. The product team needs to enable this service by updating the value of the auth status key for the SOS client in the config file to "true". This can be done directly by the product application by being a part of the 'apm' group and calling Application Permissions Manager APIs.

3.3.2.2 Detailed Instructions

References to Application Permissions Manager APIs and types that can be found in this section, for more details on these APIs and types see the Doxygen links in the following table:

API/Type Description	Doxygen Path
Register for authorization change notifications.	Modules |-> Security |-> Application Permissions Manager (APM) |-> **tpAPM_registerAuthorizationStatusChange**
Deregister for authorization change notifications.	Modules |-> Security |-> Application Permissions Manager (APM) |-> **tpAPM_deregisterAuthorizationStatusChange**
Modifies the authorization status for one or more applications.	Modules |-> Security |-> Application Permissions Manager (APM) |-> **tpAPM_setAuthorization**

3.3.2.2.1 Update an application's authorization

Permission and authorization in the cfg.json can be updated using the following APIs directly by clients given that they are registered with 'apm' group. As shown in the figure below,

© Continental Automotive GmbH 2019
Confidential and Proprietary

- The client needs to first call `tpAPM_registerAuthorizationStatusChange(const tpAPM_AuthorizationStatusCallback_t authorizationStatusCallback)` to receive change notifications.
- Thereafter if any change of authorization policy is necessary, it needs to call `tpAPM_setAuthorization(GList* applicationAuthValues)`.
- Once this change is successful, all registered clients will receive a change notification from APM.
- Also, the client can call `tpAPM_deregisterAuthorizationStatusChange()` to stop receiving any authorization status change notifications.

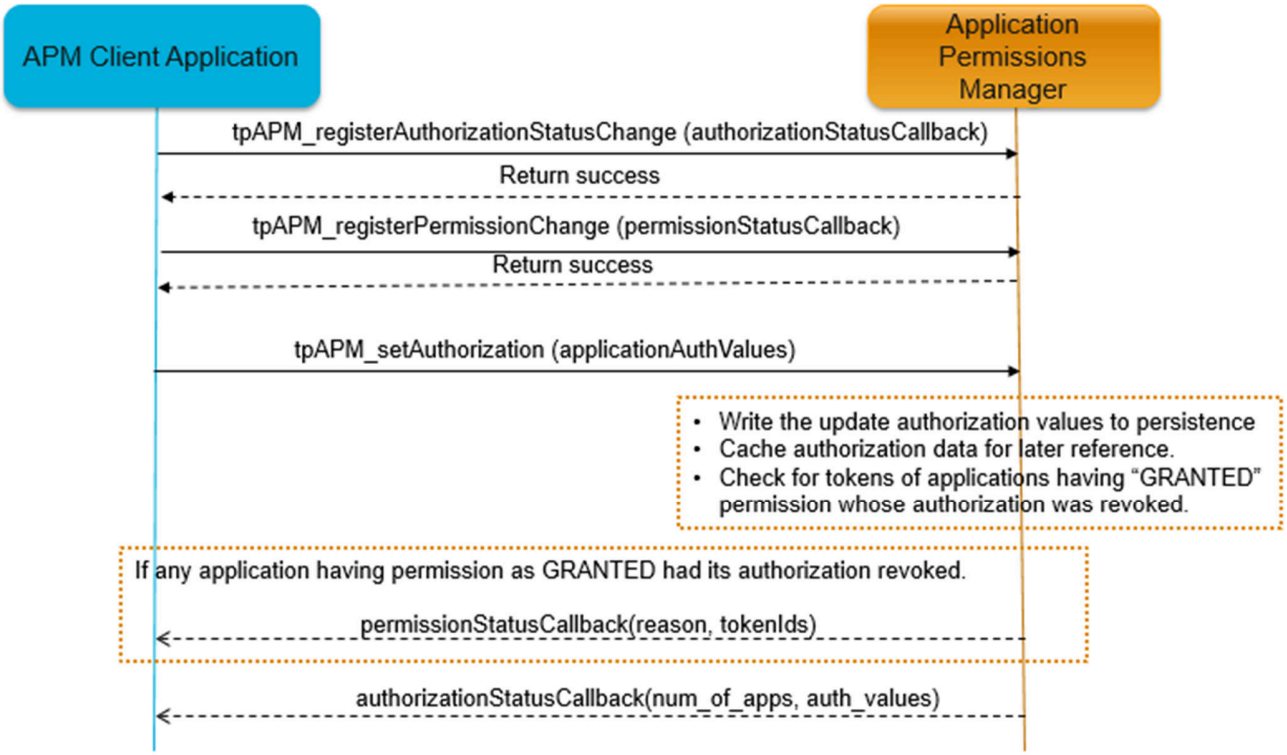


Figure 7: Update authorization status

NOTE: The `tpAPM_setAuthorization` API is prescribed for updating authorization status. If authorization status is updated directly through Persistence, then authorization status change notification will not be triggered.

3.3.2.3 Testing/Error Reporting

Not applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

4 Known Issues

- The token is not secured at this phase of the project. The Application Permissions Manager assumes that the applications requesting permissions are trusted applications and will obey and not modify the returned permissions.
- **If authorization status is updated directly through Persistence, then authorization status change notification will not be triggered.**

5 FAQs

Not Applicable.

6 Troubleshooting/Tips and Tricks

Link failure - with `CAS_APM_REMOTE_SHARED_CONFIGURATION` flag enabled:

When the link between processors breaks, clients receive a notification from Service Monitor telling them that APM service is "*NOT READY".* In that case, clients will not have the possibility to call APIs or receive notifications from APM service.

When the link is UP again, clients receive the APM service "*READY*" notification and must **implement the following sequence** to be sure their permissions are up-to-date:

- Call `tpAPM_init` and registration APIs (see TP_F101U-00023 for this requirement).
- Call `tpAPM_checkPermissions` API.
- If this check returns false, call `tpAPM_updatePermissions` API.
- If needed, call `tpAPM_releasePermissions` API for each related token id.

During link failure, clients on the Secondary AP requesting permission for a resource that is already *GRANTED* to a Primary AP client, and with a higher priority, will get a *GRANTED_ON_RELEASE* status and will have to wait until the link goes up again so that the concerned Primary AP client checks and releases its permission.

NOTE: The recovery sequence detailed above is **also applicable in case of a local APM service crash** when using APM in the "exclusive" resources mode (`CAS_APM_REMOTE_SHARED_CONFIGURATION` flag disabled), meaning an APM daemon is running independently in each processor.

7 References

7.1 Abbreviations/Acronyms/Definitions

Application/Service/ Client: These terms can be used interchangeably. It is a unit, which is product specific that runs in the background waiting to be used or carry out essential tasks. They can access the platform through the exposed functionalities. For example: Diagnostics.

Resources: These are logical entities on which permissions can be taken and be given. For example: Voice, Packet Data.

Also, note that resources can be shared if their priorities are equal.

Policies: Set of rules that govern the access of resources by clients.

Priorities: These are usually positive integers that acts as an index to determine whether a client can be given permission to access a certain resource or not. These are not Linux priorities. They start with 1, where higher

© Continental Automotive GmbH 2019
Confidential and Proprietary

priorities have lower integers. While there is only one exception of '-1' priority, indicating the client having such priority can always apply the resource successfully to permission determination, without affecting the system's current status.

Permissions: This is the type of access a client is offered by the Application Permissions Manager, which allows the client to manage the resource. Hence, APM has two types of permissions to offer to any client (based on the policies) – GRANTED and NOT GRANTED.

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary

TP_F101U-00023: User Guide Common Library

7.3 Website links

http://www.json.org/

© Continental Automotive GmbH 2019
Confidential and Proprietary

8 Revision History

Version	Date	Description
01.00	Aug 14, 2019	Initial release
02.00	Jun 29, 2020	Replace tpSYS service notify API with tpSVCMON API
03.00	Mar 16, 2021	Enable APM remote shared configuration for dual core architecture
		== ConMod-specific content from here ==

© Continental Automotive GmbH 2019
Confidential and Proprietary